

Frontend Interview Resource (JS + React + Node.js for Full Stack)

This guide is focused on topics that are commonly tricky in interviews. It includes explanations, output-based questions, code snippets, and practical patterns.

1) JavaScript Runtime Model: Engine, Call Stack, Task Queues

1.1 JS Engine and Runtime

JavaScript engine (V8, SpiderMonkey, JavaScriptCore):

- Parses source code.
- Compiles/interprets code.
- Executes on a call stack.
- Optimizes hot code paths.

Runtime (browser or Node.js):

- Provides APIs outside language core (timers, fetch, DOM, fs, sockets).
- Owns event loop and task scheduling integration.

Interview line:

- ECMAScript defines language semantics; runtime defines platform APIs.

1.2 Call Stack, Microtasks, Macrotasks

Call stack:

- Synchronous function execution.

Microtask queue:

- Promise callbacks, queueMicrotask, MutationObserver.
- Drained after current stack frame and before next macrotask.

Macrotask queue:

- setTimeout, setInterval, I/O callbacks, UI events (platform dependent details).

Important order:

1. Run sync code.
2. Drain all microtasks.
3. Run one macrotask.
4. Repeat.

1.3 Output Question 1

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
```

Expected output:

- A
- D
- C
- B

Why:

- Sync first (A, D), then microtasks (C), then macrotasks (B).

1.4 Output Question 2 (Nested Microtask)

```
setTimeout(() => {  
  console.log("T1");  
  Promise.resolve().then(() => console.log("P-in-T1"));  
}, 0);  
  
Promise.resolve().then(() => {  
  console.log("P1");  
  setTimeout(() => console.log("T2"), 0);  
});  
  
console.log("S");
```

Expected output:

- S
- P1
- T1
- P-in-T1
- T2

2) Node.js: Process, Threads, Event Loop, File Operations

2.1 JS vs Node.js

JavaScript:

- Language specification.

Node.js:

- Runtime for executing JS outside browser.
- Provides modules for fs, net, http, stream, child_process.
- Uses event loop + libuv + thread pool for some async operations.

2.2 Process vs Thread in Node.js Context

process:

- OS-level running program instance.
- Has memory space, pid, environment.

thread:

- Execution unit inside process.

Node behavior:

- Main JS execution thread runs event loop.
- libuv thread pool handles certain operations (fs, crypto, dns, zlib).
- Worker Threads can be used for CPU-bound JS tasks.

2.3 Event Loop Phases (Node high level)

Common interview framing:

- timers.
- pending callbacks.
- idle/prepare.
- poll.
- check.
- close callbacks.

Special queues:

- process.nextTick queue (runs before Promise microtasks in Node-specific behavior).
- microtask queue (Promises).

2.4 nextTick vs Promise Question

```
Promise.resolve().then(() => console.log("promise"));
process.nextTick(() => console.log("nextTick"));
console.log("sync");
```

Expected output in Node.js:

- sync
- nextTick
- promise

2.5 File Operations: Buffering vs Streaming

Small file read (simple):

```
const fs = require("fs");

fs.readFile("./data.json", "utf8", (err, data) => {
  if (err) throw err;
  console.log(data.length);
});
```

Large file read (streaming):

```
const fs = require("fs");

const stream = fs.createReadStream("./huge.log", { encoding: "utf8" });
stream.on("data", chunk => {
  // Process chunk without loading full file in memory
});
```

```
    console.log("chunk size", chunk.length);
  });
  stream.on("end", () => console.log("done"));
  stream.on("error", err => console.error(err));
```

Interview point:

- Streaming is memory-efficient and backpressure-aware.

3) Closures, Scope, and Hoisting

3.1 Closure Basics

Closure = function + lexical environment.

```
function makeCounter() {
  let count = 0;
  return function () {
    count += 1;
    return count;
  };
}

const c = makeCounter();
console.log(c()); // 1
console.log(c()); // 2
```

3.2 Common Closure Trap (Loop)

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Output:

- 3
- 3
- 3

Fix with let:

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Output:

- 0
- 1
- 2

3.3 Hoisting Clarified

- var declarations are hoisted and initialized with undefined.
- let and const are hoisted but in temporal dead zone until declaration line.
- function declarations are hoisted with function body.

Question:

```
console.log(a);  
var a = 10;
```

Output:

- undefined

Question:

```
console.log(b);  
let b = 10;
```

Output:

- ReferenceError

4) Promises, Async/Await, and Variants

4.1 Promise States

- pending.
- fulfilled.
- rejected.

4.2 Promise APIs

Promise.all:

- fails fast on first rejection.

Promise.allSettled:

- waits for all and returns status list.

Promise.race:

- settles on first settled promise.

Promise.any:

- resolves on first fulfillment; rejects if all reject.

Example:

```
const p1 = Promise.resolve("A");  
const p2 = Promise.reject(new Error("B"));  
const p3 = new Promise(res => setTimeout(() => res("C"), 50));
```

```
Promise.allSettled([p1, p2, p3]).then(console.log);
```

4.3 async/await Interview Pitfalls

Pitfall 1: sequential awaits where parallelism needed.

Bad:

```
const a = await fetchA();  
const b = await fetchB();
```

Better:

```
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

Pitfall 2: forgetting try/catch for awaited rejection.

5) Frequently Asked Output Questions (JS)

5.1 this Binding

```
const obj = {  
  x: 10,  
  getX() {  
    return this.x;  
  }  
};  
  
const fn = obj.getX;  
console.log(fn());  
console.log(obj.getX());
```

Expected output (non-strict global context may vary):

- undefined (or global x value if exists)
- 10

5.2 Arrow Function this

```
const obj = {  
  x: 42,  
  normal() { return this.x; },  
  arrow: () => this.x  
};  
  
console.log(obj.normal());  
console.log(obj.arrow());
```

Output:

- 42
- not 42 (lexical this, often undefined in modules)

5.3 Promise Chain Output

```
Promise.resolve(1)
  .then(x => x + 1)
  .then(x => { throw new Error("boom"); })
  .catch(() => 99)
  .then(x => console.log(x));
```

Output:

- 99

6) Polyfills and Utility Implementations

6.1 Throttle Polyfill

```
function throttle(fn, wait) {
  let lastTime = 0;
  let timeout = null;
  let lastArgs;

  return function throttled(...args) {
    const now = Date.now();
    const remaining = wait - (now - lastTime);
    lastArgs = args;

    if (remaining <= 0) {
      if (timeout) {
        clearTimeout(timeout);
        timeout = null;
      }
      lastTime = now;
      fn.apply(this, args);
    } else if (!timeout) {
      timeout = setTimeout(() => {
        lastTime = Date.now();
        timeout = null;
        fn.apply(this, lastArgs);
      }, remaining);
    }
  };
}
```

6.2 Debounce Polyfill

```
function debounce(fn, wait) {
  let timeout;
```

```

return function debounced(...args) {
  clearTimeout(timeout);
  timeout = setTimeout(() => fn.apply(this, args), wait);
};
}

```

6.3 Promise.all Polyfill (Simplified)

```

function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let completed = 0;

    if (promises.length === 0) {
      resolve([]);
      return;
    }

    promises.forEach((p, i) => {
      Promise.resolve(p)
        .then(value => {
          results[i] = value;
          completed += 1;
          if (completed === promises.length) {
            resolve(results);
          }
        })
        .catch(reject);
    });
  });
}

```

7) React Core: Reconciliation, Rendering, and Performance

7.1 React Reconciliation

Reconciliation:

- Process of comparing previous and next virtual tree and updating minimal real DOM parts.

Keys matter:

- Stable keys prevent unnecessary remounts and state loss.

Bad:

```
{items.map((item, index) => <Row key={index} item={item} />)}
```

Better:


```
{items.map(item => <Row key={item.id} item={item} />)}
```

7.2 Class Components to Functional Components Evolution

Then:

- Class lifecycle methods (componentDidMount, componentDidUpdate, componentWillUnmount).

Now:

- Functional components + hooks.
- Better logic reuse via custom hooks.

Interview narrative:

- Shift reduced boilerplate and improved composition.

7.3 React Hooks Types

Common hooks:

- useState.
- useEffect.
- useMemo.
- useCallback.
- useRef.
- useContext.
- useReducer.
- useLayoutEffect.
- useImperativeHandle.
- useTransition.
- useDeferredValue.
- useId.
- useSyncExternalStore.

Pattern question:

- useMemo/useCallback should optimize measured bottlenecks, not used blindly.

7.4 useEffect Tricky Cases

Infinite loop case:

```
useEffect(() => {  
  setCount(count + 1);  
}, [count]);
```

Fix by conditional logic or different state strategy.

Stale closure case:

```
useEffect(() => {  
  const id = setInterval(() => {  
    setCount(c => c + 1);  
  }, 1000);  
}, [count]);
```

```
return () => clearInterval(id);
}, []);
```

8) React Server Components and Modern Rendering

8.1 CSR, SSR, SSG, ISR (High level)

CSR:

- Rendering in browser, heavier client JS.

SSR:

- Server renders HTML per request.

SSG:

- Build-time pre-rendering.

ISR:

- Incremental regeneration for fresh content.

8.2 React Server Components (RSC)

Core idea:

- Some components render on server and reduce client JS bundle.
- Better data fetching locality for server-side concerns.

Interview point:

- Use client components for interactive UI and browser APIs.

9) Handling Large Lists and Files in Frontend

9.1 react-window for Large Data Rendering

Problem:

- Rendering 100k rows causes slow paint and memory pressure.

Solution:

- Virtualization renders only visible rows.

Example:

```
import { FixedSizeList as List } from "react-window";

function Row({ index, style, data }) {
  return <div style={style}>{data[index].name}</div>;
}

export default function BigList({ items }) {
  return (
    <List
      height={500}
    />
  );
}
```

```

    itemCount={items.length}
    itemSize={35}
    width={800}
    itemData={items}
  >
    {Row}
  </List>
);
}

```

9.2 Displaying Large Files in UI

Best practices:

- Stream on backend when possible.
- Chunk and paginate on frontend.
- Virtualize rendered lines.
- Avoid keeping entire transformed dataset in memory.

10) JS vs Node.js for Backend

JavaScript language:

- syntax, types, closures, async primitives.

Node.js backend platform:

- HTTP servers, fs operations, streams, process management, sockets.
- Non-blocking I/O model suited for many I/O-heavy APIs.

Interview trade-off:

- Node is great for high-concurrency I/O workloads.
- CPU-heavy workloads often require worker threads or separate services.

11) Sockets and Chat System Basics

11.1 WebSocket Basics

- Full-duplex persistent connection.
- Lower overhead vs repeated HTTP polling.

11.2 Chat Architecture (Interview level)

Components:

- WebSocket gateway.
- auth and session mapping.
- message broker (Redis pub/sub, Kafka, RabbitMQ depending pattern).
- persistence store for history.
- delivery ack and retry semantics.

11.3 Minimal Socket.IO Example

Server:

```
const { Server } = require("socket.io");
const io = new Server(3000, { cors: { origin: "*" } });

io.on("connection", socket => {
  socket.on("chat:msg", msg => {
    io.emit("chat:msg", { id: Date.now(), text: msg });
  });
});
```

Client:

```
import { io } from "socket.io-client";
const socket = io("http://localhost:3000");

socket.on("chat:msg", msg => console.log(msg));
socket.emit("chat:msg", "hello");
```

12) Frontend Interview Question Bank (Detailed)

12.1 JavaScript Fundamentals and Tricky Behavior

1. Explain event loop with a concrete execution order example.
2. Difference between microtask and macrotask with real callbacks.
3. var vs let vs const under hoisting and block scope.
4. Explain closure with practical use case.
5. this binding rules in regular, arrow, and bound functions.
6. call vs apply vs bind.
7. Deep copy vs shallow copy strategies and caveats.
8. Explain prototypal inheritance and **proto** vs prototype.
9. Why is [] + {} different from {} + [] in some contexts?
10. Explain temporal dead zone.

12.2 Async and Promise Questions

11. Promise.all vs allSettled vs race vs any.
12. How to cancel async work in browser and Node.
13. Why async function always returns Promise.
14. Error handling in promise chains vs async/await.
15. Write retry with exponential backoff helper.

12.3 Node.js Backend-Focused Questions

16. How Node handles concurrency with single-threaded JS.
17. What operations use libuv thread pool.
18. How to handle CPU-heavy tasks in Node.
19. Streams and backpressure explanation.
20. process.nextTick vs setImmediate vs setTimeout.
21. Explain memory leak sources in Node services.
22. How to design file upload/download pipeline safely.

12.4 React Core Questions

23. Explain reconciliation and key stability.

- 24. When does React rerender and how to avoid waste.
- 25. useMemo vs useCallback with examples.
- 26. controlled vs uncontrolled components.
- 27. useEffect lifecycle equivalent in classes.
- 28. stale closure issue and fixes.
- 29. context API performance pitfalls.
- 30. useReducer vs useState decision points.

12.5 Modern React and Architecture Questions

- 31. SSR vs CSR vs SSG vs ISR.
- 32. React server components: benefits and constraints.
- 33. Suspense and concurrent rendering basics.
- 34. Code splitting and bundle optimization.
- 35. State management choices: Redux, Zustand, context, query caches.

12.6 Performance and Scaling Questions

- 36. How to optimize huge lists/tables.
- 37. How to profile React render performance.
- 38. Preventing unnecessary re-renders in large apps.
- 39. Handling large file previews in browser.
- 40. Caching and prefetching strategies.

12.7 System Design Style Frontend Questions

- 41. Design chat frontend with typing indicators and read receipts.
- 42. Design notification system with reliable delivery semantics.
- 43. Design collaborative editor presence model at high scale.
- 44. Design dashboard with live metrics and low UI lag.
- 45. Design resilient frontend for flaky network users.

13) Practical Coding Snippets Frequently Asked

13.1 Flat Function

```
function flat(arr) {  
  const out = [];  
  for (const x of arr) {  
    if (Array.isArray(x)) {  
      out.push(...flat(x));  
    } else {  
      out.push(x);  
    }  
  }  
  return out;  
}
```

13.2 Group By Utility

```
function groupBy(list, keyFn) {  
  return list.reduce((acc, item) => {
```

```
const k = keyFn(item);
if (!acc[k]) acc[k] = [];
acc[k].push(item);
return acc;
}, {});
}
```

13.3 once Utility

```
function once(fn) {
  let called = false;
  let result;
  return function (...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);
    }
    return result;
  };
}
```

14) Final Interview Drill (Frontend)

1. Solve 10 output questions and explain each line.
2. Implement throttle, debounce, Promise.all polyfill from memory.
3. Explain React reconciliation with key examples.
4. Explain one large-list optimization using react-window.
5. Explain Node event loop and process.nextTick edge behavior.
6. Explain one websocket chat design end-to-end.

If you can do the six drills above cleanly, your frontend/full-stack interview confidence will rise significantly.